

APUNTES-PROGRAMACION.pdf



carlaserracant



Fundamentos de la Programación



1º Grado en Ingeniería del Software



**Escuela Técnica Superior de Ingeniería Informática
Universidad de Málaga**

BUCLES

Un bucle lo queremos para ejecutar ciertas líneas de código un número determinado/indeterminado de veces.

A la hora de definir un bucle tenemos que definir 3 propiedades:

1. La variable que controla cuantas veces llevo ejecutando lo que queramos o la variable que comparo en la condición de salida.
2. La condición que controla que no nos pasemos del número de veces que queremos que se haga algo (CONDICIÓN DE CONTROL)
3. La instrucción que mueve esa variable para que el bucle tenga fin. Si no controlamos bien esto obtendremos un BUCLE INFINITO.

TIPOS DE BUCLES SEGÚN USO

- **BUCLE DETERMINISTA:** Es un bucle que se hará un número determinado de veces PASE LO QUE PASE, es decir, no puede pasar nada en medio del bucle que nos haga salirnos, ni depende del estado de alguna variable, ni de algo que vayamos escribiendo. Al ejecutarse siempre un número de veces que conocemos (ej.: ejecuta esto tantas veces como una variable n me diga), se utiliza FOR porque me permite definir las 3 propiedades en la MISMA LÍNEA. Al estar todo en la misma línea facilita su comprensión a la hora de que alguien de fuera pueda ver con facilidad cuántas veces se hace tu bucle.

¡OJO! Al definir la variable “contador” DENTRO del FOR, cuando se termine el FOR esa variable será DESTRUIDA.

- **BUCLE NO DETERMINISTA:** Es un bucle en el que no puedo saber cuántas veces se va a hacer simplemente mirando alguna variable o un valor determinado. Depende de algo que vaya sucediendo DENTRO DEL BUCLE, o en los casos que quiera MÁS DE UNA CONDICIÓN DE SALIDA.

BUCLE DETERMINISTA:

```
// BUCLE FOR
for( propiedad_1; propiedad_2; propiedad_3){
    Cuerpo del bucle
}
```

BUCLE NO DETERMINISTA:

```
//BUCLE WHILE
propiedad_1;
while( propiedad_2){
    Cuerpo del bucle
    propiedad_3;
}

propiedad_1;
do{
    Cuerpo del bucle
    propiedad_3;
}while( propiedad_2);
// OJO! Esta es la unica estructura que DEBE ACABAR EN ;
```

SEMEJANZA ENTRE LOS BUCLES

Escribe un programa que calcule la suma de los N primeros números enteros positivos (el número N se leerá por teclado).

Implementa dicho programa utilizando cada una de las tres estructuras de iteración de C++: while, do while y for.

1. Creo una variable donde iré acumulando las sumas ($\text{resultado} = 1 + 2 + 3 + \dots + n$)
2. Creo un contador (`int contador = 1`) para controlar cuántas sumas tengo que hacer.
3. Leo el número (`cin >> numeroDeVeces`) (puedo controlar que cumpla cierta condición con un do...while o lectura adelantada)
4. Acumulo contador en el resultado (`resultado += contador`)
5. Incremento contador (`contador = contador + 1 / contador++`)
6. Vuelvo al paso 4 MIENTRAS que `contador <= numeroDeVeces`.

```
int resultado = 0;
int contador = 1;
cout << "Introduzca un numero: ";
cin >> numeroDeVeces;

for( int contador = 1; contador <= numeroDeVeces; contador++){
    resultado += contador;
}

while( contador <= numeroDeVeces ){
    resultado += contador;
    contador++;
}

do{
    resultado += contador;
    contador++;
}while( contador <= numeroDeVeces);
```

La estructura DO...WHILE es similar al WHILE, usaremos esta en vez de la anterior cuando queramos evaluar la condición de control a POSTERIORI de hacer lo que sea (cuando sabemos que mínimo se va a ejecutar UNA VEZ).

EJEMPLO BUCLES DETERMINISTAS

EJEMPLO 1:

Haz un bucle que muestre los primeros n números.

```
for (int contador = 1; contador <= n; contador++){
    cout << contador << " ";
}
```

EJEMPLO 2:

Escribe un programa que lea un número natural N por teclado y dibuje un triángulo de asteriscos con base y altura N.

1. Pido el número de filas que quiere que imprima (numFilas).

2. Hago un bucle que se hará tantas veces como me diga numFilas donde dibujare espacios, asteriscos y un endl, representara a cada fila que dibujo.
3. Para cada línea necesito un bucle que me dibuje un número determinado de espacios.
4. Un número determinado de asteriscos.
5. Y un endl.

```
do{
    cout << "Introduce numero de filas: ";
    cin >> numFilas;
}while( numFilas <= 0);
for(int i = 1; i <= numFilas; i++){
    for(int espacios = 1; espacios <= numFilas - i; espacios++){
        cout << " ";
    }
    for( int asteriscos = 1; asteriscos <= i; asteriscos++){
        cout << "* ";
    }
    cout << endl;
}
```

Para las instrucciones 3 y 4 me basare en la fila por la que voy y en el numero de filas totales que tengo que dibujar para saber cuantos espacios/asteriscos dibujar en cada línea.

EJEMPLO 3:

Diseña e implementa en C++ el programa Fibonacci que lea un número natural n por teclado (mayor que 0) y calcule el número de Fibonacci n -ésimo la serie. Los dos primeros números de esta serie son el cero y el uno, y a partir de éstos cada número se calcula realizando la suma de los dos anteriores.

$$Fib = 0 + 1 + 2 + \dots + n$$

Si tengo que ir sumando los dos números anteriores necesito dos variables donde guardar los dos anteriores todo el rato (a0, a1).

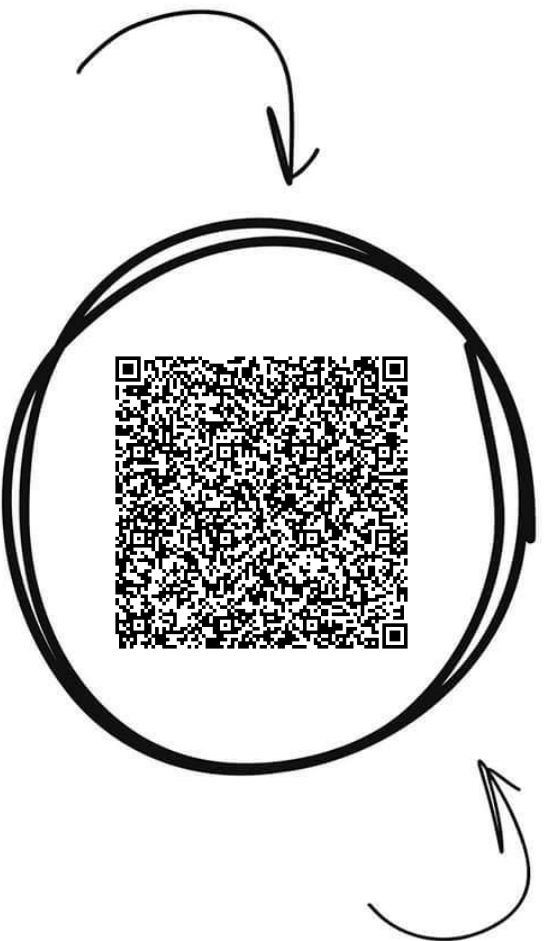
1. Establezco el a0 en 0 y el a1 en 1 (int a0 = 0 ; int a1 = 1)
2. Pido n por teclado asegurándome de que sea mayor que 0.
3. Si n es 1 o 2, ya sé la respuesta (a0 o a1)
4. Si n es mayor, debo empezar a realizar sumas.
5. Realiza la siguiente suma para generar el siguiente par (a0 , a1). Para realizar esta suma crearé un auxiliar para no machacar cualquiera de las variables a0/a1.
6. Volver al paso 5 hasta que no haga la cantidad de sumas que quiero en base a mi variable n.
7. Cuando salga dependiendo de cuantas sumas le haya dicho que haga el resultado estará en a0, a1, o en la siguiente suma (en el ejemplo lo dejaremos en a1).

```
int n;
int a0 = 0;
int a1 = 1;
do{
    cout << "Introduce n: ";
    cin >> n;
}while( n <= 0);
cout << "Resultado: ";
if( n == 1 ){
    cout << a0;
}else if( n == 2 ){
    cout << a1;
}else{
    for( int i = 2; i < n; i++){
        int aux = a0;
        a0 = a1;
        a1 = a1 + aux;
    }
    cout << a1;
}
```

Fundamentos de la Programación



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

1

Imprime esta hoja

2

Recorta por la mitad

3

Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

4

Llévate dinero por cada descarga de los documentos descargados a través de tu QR



EJEMPLO 4:

Diseña el programa tablero, en c++, que muestre por pantalla un tablero de ajedrez, donde las posiciones blancas serán mostradas con el carácter 'B' y las posiciones negras serán mostradas con el carácter 'N'. Un tablero de ajedrez tiene 8 filas y 8 columnas.

1. Hago un bucle que se ejecute 8 veces para generar líneas con B/N y un endl al final.
2. Hago un bucle que se ejecute 8 veces puesto que tengo que imprimir 8 caracteres B/N.
3. En base a en qué fila estoy y en qué columna decidiré si tengo que imprimir B/N.

```
for( int fila = 1; fila <= 8; fila++)
{
    for(int columna = 1; columna <= 8; columna++)
    {
        if( (fila + columna) % 2 == 0)
        {
            cout << "B ";
        }
        else
        {
            cout << "N ";
        }
    }
    cout << endl;
}
```

EJEMPLO DE BUCLES NO DETERMINISTAS

Es un bucle en el que no puedo saber cuántas veces se va a hacer simplemente mirando alguna variable o un valor determinado. Depende de algo que vaya sucediendo DENTRO DEL BUCLE, o en los casos que quiera MÁS DE UNA CONDICIÓN DE SALIDA.

EJEMPLO 5 (DEPENDE DE ALGO QUE YO VAYA INTRODUCIENDO)

Realizar un programa que lea caracteres alfanuméricos (letras y números) de teclado hasta que el usuario ponga un 0, y devuelva el número de letras que hay, el número de números y el número de resto de caracteres.

```
char caracter;
int contadorLetras = 0;
int contadorNumeros = 0;
int contadorResto = 0;
cout << "Introduce letras y numeros (terminado en un 0): ";
cin >> caracter;
while( caracter != '0'){
    if( (caracter >= 'a' && caracter <= 'z') || (caracter >= 'A' && caracter <= 'Z')){
        contadorLetras++;
    }else if ( caracter >= '0' && caracter <= '9'){
        contadorNumeros++;
    }else{
        contadorResto++;
    }
    cin >> caracter;
}
```

IMPORTANTE, volver a poner `cin>>caracter` dentro del bucle. ¡Recuerda! El `cin.get()` lee también los espacios, solo para caracteres.

EJEMPLO 6 (DEPENDENCIA DEL CONTENIDO DE UNA VARIABLE)

Contar la cantidad de cifras que tiene un número que se ha recogido por teclado.

```
int numero;
int cifras = 0;
cout << "Introduce el numero: ";
cin >> numero;

while( numero != 0){
    cifras++;
    numero = numero / 10;
}
```

Mostrar los *n* primeros números que cumplan que sean divisibles entre 2, 3, 5

```
int cantidadNumeros;
int posibleNumero = 2;
int contador = 0;
cout << "Cuantos numeros quieres?: ";
cin >> cantidadNumeros;
while( contador < cantidadNumeros){
    if( posibleNumero % 2 == 0 && posibleNumero % 3 == 0 && posibleNumero % 5 == 0){
        cout << posibleNumero << " ";
        contador++;
    }
    posibleNumero++;
}
```

EJEMPLO 7 (DOS CONDICIONES DE SALIDA)

Escribir un programa que adivine un número aleatorio entre 1 y 100. Cuando no acierte debe de indicar si el número es mayor o menor al aleatorio generado. Tiene 5 intentos como máximo. Si no acierta en esos intentos el jugador habrá perdido y se le mostrara el mensaje correspondiente. Para generar el número aleatorio se añadirán 2 librerías (`ctime`, `cstdlib`) y se añadirán 3 instrucciones al principio del programa.

1. Capturo el primer número (me aseguro de que esté en el rango posible)
2. Comprueba si NO es mi número y me quedan intentos para seguir en el bucle.
3. Imprimo si el número es mayor o menor que el que estoy buscando.
4. Recojo otro número y cuento un intento más.

```

int num_secreto,n,intentos=1;
srand(time(0));
num_secreto = (rand() % 100) + 1;
cout << num_secreto << endl;
do{
    cout << "\nIntroduce un numero entre 1 y 100: ";
    cin >> n;
}while( n < 1 || n > 100);

while( n != num_secreto && intentos < INTENTOS_TOTALES){

    if(n > num_secreto)
    {
        cout<<"El valor es mas pequeño. ";
    }
    else if (n < num_secreto)
    {
        cout<<"El valor es mas grande. ";
    }
    // Recojo otro intento y ME ASEGURO QUE ESTE ENTRE 1 Y 100, el mismo do...while que al principio.
    do{
        cout << "Introduce otro valor: ";
        cin >> n;
    }while( n < 1 || n > 100);
    intentos++;
}

if( intentos <= INTENTOS_TOTALES){
    cout << "Has ganado con " << intentos << " intentos.";
}else{
    cout << "Game over!, has consumido los 5 intentos.";
}

```

5. Vuelvo al paso 2.
6. Muestro si he ganado.

EJEMPLO 8 (DOS CONDICIONES DE SALIDA)

Desarrollar un algoritmo para el siguiente juego:

El usuario introduce un límite inferior, un límite superior y piensa un número en ese rango. El ordenador tiene que acertarlo. Para ello el ordenador propone un número y el usuario responde con >, <, o = (correspondiente a acertado y el programa acaba). Si la respuesta es > o <, el ordenador propondrá otro número hasta que lo acierte. Tiene sólo 5 intentos.

1. Pedir el límite inferior y superior y asegurarme de que hay números en el rango (limiteInferior...limiteSuperior).
2. Ofrecerle el número que pensamos que puede ser.
3. Recojo la respuesta del usuario.
4. En base a la respuesta modifico mis límites.

5. Contar un intento más.
6. Volver al paso 2 MIENTRAS que no haya acertado y lleve menos de 5 intentos.
7. Cuando salgo del bucle pregunto si he salido por haber acertado o porque he llegado a 6 intentos.

```

int intento = 1;
char caracter;
int numeroPropuesto;
do{
    cout << "Introduce limite inferior y superior: ";
    cin >> limiteInferior >> limiteSuperior;
}while( limiteSuperior <= limiteInferior);

do{
    do{
        numeroPropuesto = (limiteInferior + limiteSuperior) / 2;
        cout << "Es " << numeroPropuesto << " tu numero? (contesta con <, > o =): ";
        cin >> caracter;
    }while( caracter != '<' && caracter != '>' && caracter != '=');
    if( caracter == '<'){
        limiteSuperior = numeroPropuesto;
    }else if( caracter == '>'){
        limiteInferior = numeroPropuesto;
    }
    intento++;
}while( caracter != '=' && intento <= 5);

if( intento <= 5){
    cout << "Has acertado al intento " << intento;
}else{
    cout << "Game over!"
}

```

Hacemos la “media” entre el límite superior e inferior. Si el número es menor a la media entre ambos límites, el límite superior pasará a ser la media de dichos límites anteriores. El bucle, al repetirse, lo entenderá de esta forma.

USOS MÁS FRECUENTES

A continuación, se mostrarán los bucles que suelen pedir en la mayoría de los ejercicios/exámenes/prácticas.

Asegurarnos que una variable que leemos cumpla cierta condición (sea positiva, no sea 0, sea par...)

Do...while

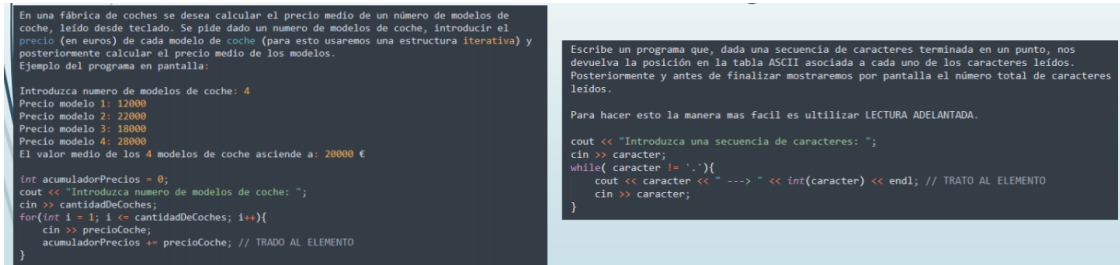
Lectura adelantada

<pre> do{ PIDO QUE ME INTRODUZCA EL DATO }while(CONDICION QUE ME DICE SI EL DATO ESTA MAL) do{ cout << "Introduzca un numero entre 1 y 10: "; cin >> numero; }while(numero < 1 numero > 10); </pre>	<pre> PIDO QUE ME INTRODUZCA EL DATO WHILE(CONDICION QUE ME DICE SI EL DATO ESTA MAL){ VUELVO A PEDIR EL DATO } cout << "Introduzca un numero entre 1 y 10: "; cin >> numero; while (numero < 1 numero > 10){ cout << "Error. Introduzca un numero entre 1 y 10: "; cin >> numero; } </pre>
---	--

Leer una serie de números/caracteres, normalmente nos lo presentan en dos situaciones.

Leo cuantos números voy a leer,
y después los leo.

Leo números mientras que lo que
haya leído no sea algo



```
En una fábrica de coches se desea calcular el precio medio de un número de modelos de coche, leído desde teclado. Se pide dado un número de modelos de coche, introducir el precio (en euros) de cada modelo de coche (para esto usaremos una estructura iterativa) y posteriormente calcular el precio medio de los modelos.
Ejemplo del programa en pantalla:
Introduzca numero de modelos de coche: 4
Precio modelo 1: 12000
Precio modelo 2: 22000
Precio modelo 3: 18000
Precio modelo 4: 20000
El valor medio de los 4 modelos de coche asciende a: 20000 €

int acumuladorPrecios = 0;
cout << "Introduzca numero de modelos de coche: ";
cin >> cantidadDeCoches;
for(int i = 1; i <= cantidadDeCoches; i++){
    cin >> precioCoche;
    acumuladorPrecios += precioCoche; // TRATO AL ELEMENTO
}

Escribe un programa que, dada una secuencia de caracteres terminada en un punto, nos devuelva la posición en la tabla ASCII asociada a cada uno de los caracteres leídos.
Posteriormente y antes de finalizar mostraremos por pantalla el número total de caracteres leídos.
Para hacer esto la manera mas facil es utilizar LECTURA ADELANTADA.

cout << "Introduzca una secuencia de caracteres: ";
cin >> caracter;
while( caracter != '.'){
    cout << caracter << " ---> " << int(caracter) << endl; // TRATO AL ELEMENTO
    cin >> caracter;
}
```

Mostrar una serie de números/caracteres que cumplan cierta condición. (Genero yo un rango de posibles valores y FILTRO solo aquellos que cumplen mi condición).

Mostrar los n primeros números que cumplan que sean divisibles entre 2, 3, 5.

El rango de valores que estoy generando aquí es [2...infinito]. Baso mi bucle en la cantidad de números que voy encontrando en ese rango que cumplen mi condición. OJO! Asegurarse de que hay números suficientes en ese rango que cumplan vuestra condición. Sino es posible que generéis infinitos números y nunca lleguéis a mostrar la cantidad de números que queréis y que nunca salgáis del bucle (BUCLE INFINITO).



```
int cantidadNumeros;
int posibleNumero = 2;
int contador = 0;
cout << "Cuantos numeros quieres?: ";
cin >> cantidadNumeros;
while( contador < cantidadNumeros){
    if( posibleNumero % 2 == 0 && posibleNumero % 3 == 0 && posibleNumero % 5 == 0){
        cout << posibleNumero << " ";
        contador++;
    }
    posibleNumero++;
}
```

Solo quiero buscar el primero que cumpla la condición en mi rango de valores

Si por ejemplo nos pidiesen solo EL PRIMER VALOR del rango de valores que cumple nuestra condición mezclaríamos los conceptos de los dos ejemplos anteriores con el de tener DOS CONDICIONES DE SALIDA que vimos con anterioridad en los ejemplos 7 y 8, ya que cuando encontremos ese valor NO QUEREMOS SEGUIR MIRANDO MÁS en nuestro rango de valores.

Hallar que dos catetos formarían mi triángulo rectángulo para una hipotenusa dada por el usuario ($a^2 = \text{incógnita}_1^2 + \text{incógnita}_2^2$).

Este ejercicio mezcla lo que estamos viendo de generar un rango de valores [1...(a-1)][1...(a-1)] con lo visto anteriormente de tener DOS CONDICIONES DE SALIDA en nuestro bucle, haciendo que si encontramos un par que cumple nuestra condición, NO SEGUIMOS BUSCANDO.

```
int a;
int i = 1;
int j;
bool encontrado = false;

cout << "Introduzca la longitud de la hipotenusa: ";
cin >> a;
while( !encontrado && i < a){
    j = i;
    while( !encontrado && j < a){
        if( pow(a,2) == pow(i,2) + pow(j,2)){
            encontrado = true;
        }else{
            j++;
        }
    }
    if( !encontrado){
        i++;
    }
}
if( encontrado){
    cout << "b: " << i << ", c: " << j << endl;
}else{
    cout << "No encontrado!!!";
}
```

El rango de valores se va generando “solo” a través de modificaciones que le voy haciendo a una variable.

Recojo un número y muestro la cantidad de cifras pares que tiene.

El rango de valores aquí se van generando solos al ir “cargándome” el número en cada iteración (numero = numero / 10) e ir cogiendo su dígito más a la derecha (numero % 10). Cada uno de esos dígitos a la derecha que voy sacando serán mis posibles valores que pueden cumplir la condición.

```
unsigned numero;
unsigned acumulador = 0;
cout << "Introduzca el numero POSITIVO: ";
cin >> numero;
do{
    if( (numero % 10) % 2 == 0){
        acumulador++;
    }
    numero = numero / 10;
}while( numero > 0);
cout << acumulador;
```

FUNCIONES

La única finalidad del uso de funciones es simplificar el trabajo y tener un mayor control de todo. Mediante las funciones, podemos localizar mejor los errores, ya que las distintas “tareas” las estructuramos en “bloques” distintos.

Cuando se produce una llamada a un subprograma:

- Se establecen las vías de comunicación entre llamante y llamado (a través de los parámetros).

- b) Se crean las variables declaradas en el llamado.
- c) El flujo de control pasa a la primera instrucción del llamado.

Cuando acaba el subprograma llamado:

- a) Se destruyen las variables creadas en el llamado.
- b) El flujo de control continúa por la instrucción que sigue a la de llamada en llamante.

En la práctica, los lenguajes de programación normalmente implementan el flujo de información mediante:

- Paso por valor:

Se almacena en el parámetro formal una copia del parámetro real. Cualquier modificación en el formal no afecta al real.

Sintaxis: TipoDato parámetro

- Paso por referencia:

Se almacena en el parámetro formal una referencia a la variable situada como parámetro real. Cualquier modificación en el formal implica una modificación en el real.

Sintaxis: TipoDato& parámetro

Parámetro formal: La caja que tiene la función de llamada donde espera que le dejen lo que sea.

Parámetro real: La caja que tiene la que va a llamar a la función, donde está el dato original.

Paso de parámetros por valor:

—Sólo permite comunicación de entrada (*ImprimirResultado*)

—Aísla.

—Permite variables, constantes y expresiones como parámetro real.

—Duplica memoria y realiza copia.

—Utiliza más memoria con los tipos estructurados.

Paso de parámetros por referencia:

—Permite comunicación de salida si el parámetro formal se utiliza para comunicar un valor al llamante (*leerDatos*) y de entrada/salida si se modifica el valor contenido en el parámetro formal (*intercambiar*).

—También permite comunicación de entrada si no hay modificación del parámetro real. Normalmente se utiliza para los tipos de datos estructurados.

- No aísla.
- Sólo permite una variable como parámetro real.
- No duplica memoria y no realiza copia.
- Utiliza menos memoria en el caso de tipos estructurados.

Si nuestra función NO VA A DEVOLVER NADA, imaginemos que recibe la mesa, se la muestra al usuario, y acaba nuestra tarea. En este caso no devolvemos nada, solo recibimos algo, hacemos algo con ello pero NO CONTESTAMOS CON NADA. En estos casos como no devuelvo nada, para especificar que nuestra función recibe cosas, pero no devuelve nada se le pone la palabra VOID en el tipo de dato devuelto. Normalmente este tipo de funciones suelen usarse para cuando quiero recibir algo e imprimirlo simplemente, no tengo que contestar con el resultado de nada, o bien contesto con algo pero lo devuelvo por PARÁMETROS POR REFERENCIA,

Ej. Quiero hacer una función que RECIBA un número y MUESTRE (mostrar != devolver) si ese número es par o no.

```
void esPar(int numero){
    if( numero % 2 == 0){
        cout << "Es par";
    }else{
        cout << "Es impar";
    }
}
```

A continuación voy a intentar ejemplificar los dos PRINCIPALES MOTIVOS POR EL QUE QUEREMOS FUNCIONES:

- Nos ayudan a no repetir código.

Ej.: Recoge dos números y muestra cuales de ellos son divisibles entre 2,3,5.

Esta sería la solución que pensaríamos antes de hacer funciones, que es lo malo aquí? que estamos repitiendo la manera de analizar uno de los números DOS VECES, y lo único que cambia en la sección A con respecto a la sección B es la variable que uso para comparar.

```
int a,b;
cout << "Introduce dos numeros: ";
cin >> a >> b;

// Sección A
if( a % 2 == 0 && a % 3 == 0 && a % 5 == 0){
    cout << a << " ";
}

// Sección B
if( b % 2 == 0 && b % 3 == 0 && b % 5 == 0){
    cout << b << " ";
}
```

Si yo posteriormente te pidiese que también quiero que sea divisible por 7, tendría que recorrer todo tu código y cada vez que apareciesen una de estas dos secciones cambiarlo EN LOS DOS SITIOS. Esto no es para nada eficiente puesto que tendremos más líneas en nuestro código (lo que lo hará más difícil de entender), además de que si tenemos que cambiarlo en todos los sitios donde aparezca lo más probable es que nos dejemos algunos, provocando otro error que quizás tardemos más en darnos cuenta.

Por tanto si quiero sacar el comportamiento de las dos secciones en una función a parte lo único que tengo que pensar es:

- Que variables necesita mi función.
- Que necesito que me conteste (el resultado de una operación, si cumple una condición o no,...).

En este caso, necesito que mi función reciba un número (a/b) y me conteste si ese número es divisible entre 2,3,5,7.

```
bool esDivisible ( int numero ){
    return numero % 2 == 0 && numero % 3 == 0 && numero % 5 == 0 && numero % 7 == 0;
}

int main(){
    int a,b;
    cout << "Introduce dos numeros: ";
    cin >> a >> b;

    if( esDivisible(a)){
        cout << "a" << " ";
    }
    if( esDivisible(b)){
        cout << "b" << " ";
    }
}
```

—Simplifica el problema en uno más pequeño, lo que me ayuda a centrarme en el problema pequeño y no en el grande (programación descendente). Además de que me permite REUTILIZAR CÓDIGO.

```
Ej_1.2: Muestra los n primeros primos (empezando desde el 2):

int cantidadNumeros;
int contador = 0;
int posiblePrimo = 2;
cout << "Cuantos primos quieres?: ";
cin >> cantidadNumeros;

while( contador < cantidadNumeros){
    // Aquí tengo que ver si posiblePrimo es primo, y en ese caso mostrarlo e incrementar el contador para contar ese primo que acabo de mostrar y parar en algún momento
    // En cualquier caso, sea primo o no tendré que avanzar al siguiente posiblePrimo para no estar siempre preguntando por el mismo.

    // Como se si un numero es primo? si no tiene divisores entre 2 y (numero - 1)

    // SECCION QUE PODEMOS SACAR

    int posibleDivisor = 2;
    bool esPrimo = true;
    while( esPrimo && posibleDivisor < posiblePrimo){
        if( posiblePrimo % posibleDivisor == 0){
            esPrimo = false;
        }
        posibleDivisor++;
    }

    // Una vez tengo en mi variable booleana "esPrimo" si el numero es primo o no, en base a eso muestro ese posiblePrimo y cuento una vez, o simplemente avanzo
    // y no hago nada mas
    if( esPrimo ){
        cout << posiblePrimo << " ";
        contador++;
    }
    posiblePrimo++;
}

Como podemos observar la seccion que analiza un numero y me comprueba si es primo o no, me emborrana un poco el codigo, ademas de que probablemente nos pueda ser util para algun otro
ejercicio, por tanto sacaremos ese comportamiento en una funcion nueva que recibirá un numero como parametro y solo se encargara de decirme si es primo o no.

bool esPrimo (int numero){
    int posibleDivisor = 2;
    bool esPrimo = true;
    while( esPrimo && posibleDivisor < posiblePrimo){
        if( posiblePrimo % posibleDivisor == 0){
            esPrimo = false;
        }
        posibleDivisor++;
    }
}

int main(){
    int cantidadNumeros;
    int contador = 0;
    int posiblePrimo = 2;
    cout << "Cuantos primos quieres?: ";
    cin >> cantidadNumeros;

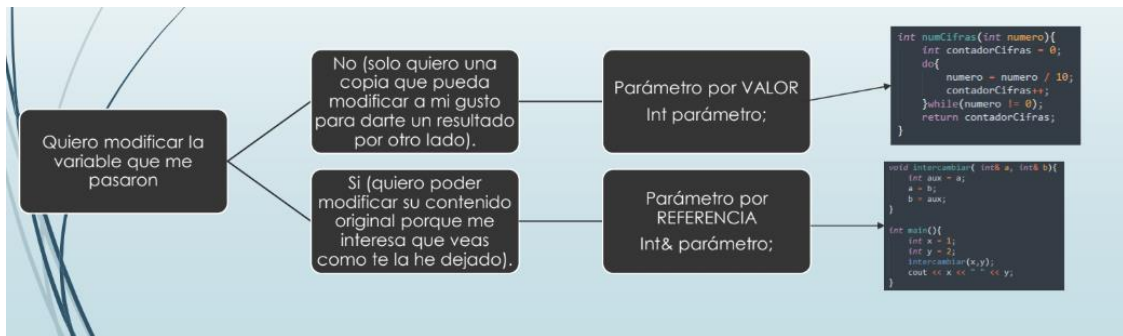
    while( contador < cantidadNumeros){
        if( esPrimo(posiblePrimo ) ){
            cout << posiblePrimo << " ";
            contador++;
        }
        posiblePrimo++;
    }
}
```

A la vista esta de que el codigo queda mucho mas limpio, entendible y que si en algun otro ejercicio tengo que ver si un numero es primo o no, bastara con usar la funcion que acabamos de definir.

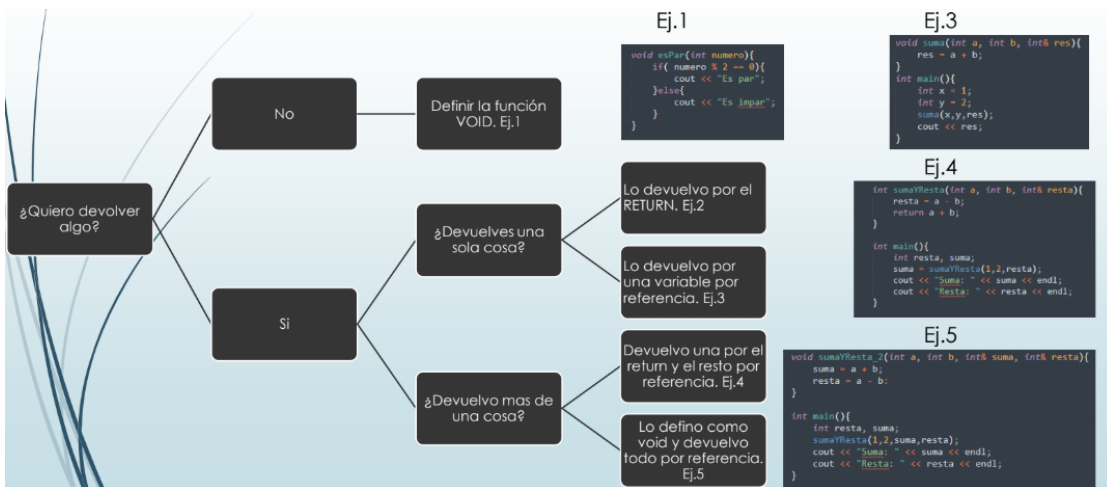
Por resumir un poco;

A la hora de plantearme como definir la función (paso de parámetros y valores devueltos), tengo que plantearme las siguientes preguntas;

A la hora de plantearme los parámetros que necesita mi función tengo que preguntarme lo siguiente para cada una de las variables que le pase.



A la hora de plantearme lo que quiero devolver:



Ej.2

```

int suma(int a, int b){
    return a + b;
}

int main(){
    int x = 1;
    int y = 2;
    int res = suma(x,y);
}
  
```